

GUÍA DEL ALUMNO · JAVASCRIPT EN EL NAVEGADOR

Vuestro ahorcado, paso a paso

Buenas, soy Ramón Abramo, profesor con más de 25 años de experiencia docente. Esta guía os lleva de la mano para programar el juego del ahorcado de tres formas, de la más sencilla a la más profesional. Un paso por diapositiva, una prueba al final de cada paso y la solución escondida por si os atascáis.



Lo que necesitáis tener abierto

1. Abrid la carpeta de la solución que vayáis a hacer (`version-1-basica-lineal`, `version-2-teclado-eventos` o `version-3-avanzada-poo`) en vuestro editor.
2. El `index.html` carga un archivo llamado `app.js`. Si os he dado la plantilla como `app.alumnos.js`, renombradla a `app.js` antes de empezar. Ese es el archivo que vais a rellenar.
3. Abrid el `index.html` en el navegador.
4. Pulsad `F12` y quedaos en la pestaña **Consola**[1]. Ahí veréis los errores en rojo y podréis probar cosas a mano.



⚠️ **Ojo:** no toquéis el `index.html` ni el `main.css`. Todo el trabajo va en `app.js`. Si cambiáis un `id` del HTML, vuestro JavaScript dejará de encontrarlo.

El ciclo de trabajo, siempre el mismo

✍️ **Escribir** → 💾 **Guardar** `Ctrl` + `S` → 🔄 **Recargar** `F5` → 🧪 **Probar**

Parece una tontería, pero la mitad de los «no funciona» que veo en clase son archivos sin guardar o páginas sin recargar.



🔍 **Para entenderlo:** el ahorcado es como un examen tipo test con un único aspirante. Hay una respuesta secreta, el aspirante va marcando letras y alguien lleva la cuenta de los errores. Vuestro código es ese «alguien».



Así está organizado cada paso



🎯 **Objetivo:** qué tiene que conseguir el paso, en una frase.

📍 **Dónde:** en qué bloque `TODO` de vuestro `app.js` se escribe.

Pistas, en orden

Lo que tenéis que escribir, contado poco a poco. Intentadlo primero solo con las pistas.



✅ **Comprobadlo:** la prueba concreta que tiene que salir bien antes de pasar al siguiente paso.



🆘 **Si no funciona:** los fallos más típicos de ese paso y cómo arreglarlos.

Al final de cada paso está la solución, escondida. Abridla solo después de intentarlo. Copiar sin entender es como ver la respuesta de un crucigrama: rellena la casilla, pero no aprendéis nada.

Cuando el paso funcione, marcad la casilla **«¡Hecho!»**. Se queda guardada en vuestro navegador y os sirve para ver cuánto os queda.



El dibujo del ahorcado está escondido desde el principio

```
<svg id="svg-ahorcado" viewBox="0 0 200 240">
  <!-- La horca: siempre visible -->
  <line ... class="trazo-horca" />

  <!-- El muñeco: una pieza por fallo -->
  <circle id="parte-cabeza" class="parte-oculta" />
  <line id="parte-cuerpo" class="parte-oculta" />
  <line id="parte-brazo-izq" class="parte-oculta" />
  <line id="parte-brazo-der" class="parte-oculta" />
  <line id="parte-pierna-izq" class="parte-oculta" />
  <line id="parte-pierna-der" class="parte-oculta" />

  <g id="parte-cara-derrota" class="parte-oculta">...</g>
</svg>
```

Fijaos en el truco: el muñeco entero ya está dibujado en el HTML, pieza a pieza. Lo que pasa es que cada pieza lleva la clase `parte-oculta` y el CSS la deja invisible.

Para «dibujar» una pieza no hay que pintar nada. Basta con añadirle la clase `visible`[2]:

```
document.getElementById('parte-cabeza').classList.add('visible');
```



🔍 **Para entenderlo:** es un teatro de sombras con todas las figuras ya colocadas detrás de la cortina. Vosotros solo vais encendiendo focos.



✅ **Comprobadlo:** pegad esa línea en la consola. La cabeza aparece al momento. Recargad y vuelve a desaparecer.



Cuatro cosas que el juego tiene que recordar

Pensad en lo que haríais jugando con papel y boli. ¿Qué apuntáis?

QUÉ RECUERDA	EJEMPLO A MEDIA PARTIDA
La palabra secreta	`'PANTALLA'`
Lo que se ve en el tablero	`['_', 'A', '_', '_', 'A', 'L', 'L', 'A']`
Las letras ya probadas	`['A', 'L', 'E']`
Los fallos que lleva	`1` (la `E`)

Y una quinta, de propina: si la partida ya ha terminado, para no seguir aceptando letras.

Las tres soluciones guardan exactamente esta información. Lo que cambia es dónde:

- **Solución 1:** en variables sueltas (`palabraSecreta` , `letrasAdivinadas` , `letrasUsadas` , `fallos`).
- **Solución 2:** igual, pero las letras usadas van en un `Set` [3], una colección que no admite repetidos.
- **Solución 3:** dentro de un objeto de la clase `AhorcadoEngine` [4], el «motor» del juego.



🔍 **Para entenderlo:** el tablero es un array del mismo tamaño que la palabra. Cada casilla empieza tapada y se destapa cuando acertáis su letra. Como un calendario de adviento.



Las piezas de JavaScript que vais a usar

Elegir al azar

```
Math.floor(Math.random()
* 8)
// → un entero del 0 al
7
```

`Math.random()` da un decimal entre 0 y 1 (sin llegar a 1)[5]; `Math.floor()` le quita los decimales[6].

¿Está dentro?

```
'PANTALLA'.includes('A')   //
true
['A', 'L'].includes('E')  //
false
```

`includes()` existe para textos[7] y para arrays[8]. Devuelve `true` o `false`.

Array → texto

```
['_', 'A', '_'].join(' ')
// → '_ A _'
```

`join()` pega los elementos de un array con el separador que le digáis[9].

Escribir en pantalla

```
elemento.textContent =
'Hola';
```

Cambia el texto de una etiqueta[10].

Clases CSS

```
el.classList.add('visible');
el.classList.remove('visible');
```

Pone y quita clases sin tocar las demás[2].

Escuchar eventos

```
boton.addEventListener('click',
jugar);
```

«Cuando pase esto, llama a esta función»[11].

ELIGE TU CAMINO



Tres soluciones, el mismo juego

Elegid la que vayáis a hacer. Mi consejo: empezad por la 1, seguid con la 2 y dejad la 3 para el final. Cada una sube un escalón respecto a la anterior.

1 · Básica lineal

Una caja de texto, un botón «Probar» y variables globales. Lo justo para entender la lógica del juego.

2 · Teclado y eventos

Un teclado en pantalla creado con JavaScript, delegación de eventos, teclado físico y pistas.

3 · Avanzada con clases

El juego repartido en clases: motor, sonido, estadísticas e interfaz. Con niveles de dificultad.



1

SOLUCIÓN 1 · CARPETA VERSION-1-BASICA-LINEAL

Básica lineal

El jugador escribe una letra en una caja de texto y pulsa «Probar» o Enter. Vosotros elegís la palabra, la tapáis con guiones, comprobáis cada letra, dibujáis el muñeco y decidís quién gana.

Ya escrito · 1 · Elegir palabra · 2 · Pintar tablero · 3 · Escuchadores · 4 · Validar · 5 · Aciertos · 6 · Fallos · 7 · Fin

SOLUCIÓN 1 · PREPARATIVOS (YA ESCRITOS)



Lo que ya tenéis en la plantilla

```
const PALABRAS = ['JAVASCRIPT', 'PROGRAMACION', ...];
const MAX_FALLOS = 6;

let palabraSecreta = '';
let letrasAdivinadas = [];
let letrasUsadas = [];
let fallos = 0;
let juegoTerminado = false;

const palabraDisplay = document.getElementById('palabra-display');
const inputLetra = document.getElementById('input-letra');
// ... y el resto de elementos

const partesAhorcado = [
  document.getElementById('parte-cabeza'),
  // ... las 6 piezas, en orden de aparición
];
```

Los bloques `TODO 1` y `TODO 2` vienen resueltos: el banco de palabras, las variables de memoria y los elementos de la página cogidos con `getElementById` [12].

Fijaos en `partesAhorcado`. Es un array con las seis piezas del muñeco **en el orden en que deben aparecer**. Así, el fallo número 1 enciende `partesAhorcado[0]`, el 2 enciende `partesAhorcado[1]`… Ese «menos uno» os va a salir más adelante.

También están escritas `mostrarMensaje()` y `finalizarPartida()`. Usadlas, no las reescribáis.



✅ **Comprobadlo:** escribid en la consola `partesAhorcado.length`. Tiene que salir `6`. Si sale algún `null` al escribir `partesAhorcado`, algún `id` está mal escrito.



Elegir la palabra y taparla con guiones



🎯 **Objetivo:** al empezar la partida, coger una palabra al azar del banco y preparar un array con un guion por cada letra.

📍 **Dónde:** bloques `TODO 3.1`, `TODO 3.2` y `TODO 3.3`, dentro de `function iniciarJuego()`.

Pistas, en orden

1. Calculad un índice al azar: `Math.floor(Math.random() * PALABRAS.length)` [5].
2. Guardad en `palabraSecreta` la palabra de esa posición.
3. Vaciad `letrasAdivinadas` y, con un bucle `for` que vaya de `0` a `palabraSecreta.length`, meted un `'_'` en cada vuelta con `push`.
4. Dejad la memoria limpia: `letrasUsadas` vacío, `fallos` a `0` y `juegoTerminado` a `false`.



🔍 **Para entenderlo:** sacáis una papeleta de una bolsa (el índice) y dibujáis en la pizarra tantas rayitas como letras tiene.



✅ **Comprobadlo:** en pantalla todavía no se ve nada, y es normal. Recargad y escribid en la consola `palabraSecreta` y luego `letrasAdivinadas`. Para `'ARRAY'` tiene que salir `['_', '_', '_', '_', '_']`. Escribid `iniciarJuego()` varias veces: la palabra debe ir cambiando.



🆘 **Si no funciona:** si `palabraSecreta` sale `undefined`, habéis multiplicado por un número fijo mayor que 7 o habéis olvidado el `Math.floor`. Si el array tiene una casilla de más, el bucle lleva `<=` en vez de `<`.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-1**. Intentadlo antes de mirar.

☐ ¡Hecho! La palabra cambia en la consola



Pintar el tablero en pantalla



🎯 **Objetivo:** que al cargar la página se vean los guiones, las 6 vidas y la lista de letras probadas, con el muñeco escondido.

📍 **Dónde:** la función `actualizarPantalla()` (`TODO 5.1` a `5.3`) y el final de `iniciarJuego()` (`TODO 3.4` a `3.6`).

Pistas, en orden

1. En `actualizarPantalla()` : `palabraDisplay.textContent = letrasAdivinadas.join(' ')` [9].
2. Las vidas son `MAX_FALLOS - fallos` . Escribidlas en `vidasRestantes` .
3. Las letras probadas, unidas con `', '` , en `letrasUsadasDisplay` . Si no hay ninguna, mejor escribid «Ninguna aún».
4. En `iniciarJuego()` , recorred `partesAhorcado` con `forEach` y quitad la clase `visible` a cada pieza. Haced lo mismo con `caraDerrota` .
5. Activad `inputLetra` y `btnProbar` (`disabled = false`), vaciad la caja y ponedle el foco.
6. Llamad a `actualizarPantalla()` y a `mostrarMensaje()` con un saludo.



✅ **Comprobadlo:** recargad. Tienen que verse los guiones separados (`_ _ _ _ _`), «Vidas restantes: 6 / 6», «Ninguna aún» y vuestro mensaje de bienvenida. El cursor debe estar ya dentro de la caja de texto.



🆘 **Si no funciona:** si los guiones salen pegados (`_____`) o con comas (`_,_,_`), os falta el `' '` dentro del `join` . ¿Por qué esconder el muñeco si ya empieza escondido? Porque `iniciarJuego()` también se usa para la segunda partida, y ahí sí puede haber piezas a la vista.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-2**. Intentadlo antes de mirar.

☐ ¡Hecho! Se ven los guiones y las 6 vidas



Que el botón y la tecla `Enter` hagan algo



🎯 **Objetivo:** que pulsar «Probar Letra» o `Enter` llame a `procesarLetra()`, y que «Nueva Palabra» llame a `iniciarJuego()`.

📍 **Dónde:** bloques `TODO 7.1`, `7.2` y `7.3`, al final del archivo.

Pistas, en orden

1. `btnProbar.addEventListener('click', procesarLetra)`[11]. Sin paréntesis detrás de `procesarLetra`: le dais la función, no la ejecutáis.
2. En `inputLetra`, escuchad `'keydown'`[13]. La función recibe el evento: si `evento.key === 'Enter'`[14], llamad a `procesarLetra()`.
3. `btnReiniciar` con `'click'` llama a `iniciarJuego`.



🔍 **Para entenderlo:** un escuchador es un timbre. Lo instaláis una vez en la puerta y, cada vez que alguien llama, suena solo. No hace falta instalarlo de nuevo en cada visita.



✅ **Comprobadlo:** escribid una letra y pulsad `Enter`: la caja se vacía y el cursor vuelve a ella (eso ya lo hace `procesarLetra()`). Pulsad «Nueva Palabra» varias veces: el número de guiones va cambiando.



🆘 **Si no funciona:** si al cargar la página ya pasa algo raro sin tocar nada, habéis escrito `procesarLetra()` con paréntesis dentro del `addEventListener`. Así se ejecuta en el acto y el escuchador recibe lo que devuelve (nada).



¿Atascados? La solución de este paso está en el anexo, apartado **S1-3**. Intentadlo antes de mirar.

☐ ¡Hecho! Enter vacía la caja



Validar la letra antes de jugarla



🎯 **Objetivo:** rechazar lo que no sea una letra y las letras repetidas, sin gastar vidas. Las válidas se apuntan en la lista de probadas.

📍 **Dónde:** bloques `TODO 4.2`, `4.3` y `4.5`, dentro de `procesarLetra()`. La lectura de la letra (`TODO 4.1`) ya está hecha con `trim()` [15] y `toUpperCase()` [16].

Pistas, en orden

1. Comprobad que es una sola letra con la expresión regular `/^[A-ZÑ]$/` y su método `test()` [17]. Si no lo es, avisad con `mostrarMensaje(..., 'aviso')` y `return`.
2. Si `letrasUsadas.includes(letra)` [8], avisad y `return`.
3. Si ha pasado los dos filtros, `letrasUsadas.push(letra)`.
4. Al final de la función: `actualizarPantalla()` y `verificarFinJuego()` (esta todavía está vacía; no pasa nada).



🔍 **Para entenderlo:** el portero de una discoteca. Primero mira el DNI (¿es una letra?) y luego la lista (¿ya ha entrado?). Si falla cualquiera de las dos, a la calle, y la fiesta sigue como estaba.



✅ **Comprobadlo:** escribid `7` y Enter: aviso, y las vidas siguen en 6. Escribid `a`: aparece `A` en «Letras probadas» (en mayúscula). Escribid otra vez `a`: aviso de repetida y la lista no cambia.



🆘 **Si no funciona:** si la `ñ` se rechaza, os falta la `Ñ` dentro de los corchetes. Si una letra repetida se apunta dos veces, el `push` está antes de la comprobación o falta el `return`.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-4**. Intentadlo antes de mirar.

☐ ¡Hecho! El 7 y las repetidas se rechazan



Destapar las letras acertadas



🎯 **Objetivo:** si la letra está en la palabra, cambiar el guion por la letra en **todas** las posiciones donde aparezca.

📍 **Dónde:** bloque `TODO 4.4`, justo después del `push` y antes de `actualizarPantalla()`.

Pistas, en orden

1. `if (palabraSecreta.includes(letra))`[7].
2. Dentro, un `for` que recorra la palabra. Si `palabraSecreta[i] === letra`, entonces `letrasAdivinadas[i] = letra`.
3. Celebradlo con `mostrarMensaje(..., 'exito')`.
4. Dejad preparado el `else` vacío: lo rellenáis en el paso siguiente.



🔍 **Para entenderlo:** el calendario de adviento otra vez: si la letra es la A, abrís todas las ventanitas que tengan una A dentro, no solo la primera.



✅ **Comprobadlo:** mirad la palabra con `palabraSecreta` en la consola (sí, es trampa; para probar vale). Si es `PANTALLA` y escribís `A`, tiene que salir `_ A _ _ A _ _ A`: las tres A de golpe.



🆘 **Si no funciona:** si solo se destapa la primera A, estáis usando `indexOf` o tenéis un `break` dentro del bucle. Si no se destapa nada, comprobad que comparáis con `===` y no con `=`.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-5**. Intentadlo antes de mirar.

☐ ¡Hecho! Se destapan todas las apariciones



Contar el fallo y dibujar una pieza



🎯 **Objetivo:** cada letra que no está suma un fallo, resta una vida y enciende la siguiente pieza del muñeco.

📍 **Dónde:** el `else` del paso anterior y la función `dibujarSiguienteParte(numeroFallo)` (`TODO 5.4`).

Pistas, en orden

1. En el `else` : `fallos++` , luego `dibujarSiguienteParte(fallos)` y un mensaje de tipo `'error'` .
2. En `dibujarSiguienteParte` : la pieza es `partesAhorcado[numeroFallo - 1]` . ¿Por qué el menos uno? Porque el primer fallo es el 1 y la primera posición del array es la 0.
3. Si esa pieza existe, `classList.add('visible')` [2].



✅ **Comprobadlo:** probad letras que no estén en ninguna palabra del banco: `K` , `W` , `Q` ... Primer fallo: cabeza y 5 vidas. Segundo: tronco y 4. Tercero y cuarto: brazos. Quinto y sexto: piernas.



🆘 **Si no funciona:** si el primer fallo dibuja el tronco en vez de la cabeza, os sobra el `- 1` o hacéis `fallos++` después de dibujar. Si las vidas no bajan, revisad que `actualizarPantalla()` sigue al final de `procesarLetra()` .



¿Atascados? La solución de este paso está en el anexo, apartado **S1-6**. Intentadlo antes de mirar.

☐ ¡Hecho! El muñeco se dibuja en orden



¿Has ganado o has perdido?



🎯 **Objetivo:** después de cada jugada, comprobar si ya no quedan guiones (victoria) o si se han agotado las vidas (derrota). En los dos casos, bloquear la partida.

📍 **Dónde:** función `verificarFinJuego()`, bloques `TODO 6.1` y `6.2`.

Pistas, en orden

1. Victoria: `!letrasAdivinadas.includes('_')`. Poned `juegoTerminado = true`, un mensaje de `'exito'` y llamad a `finalizarPartida()`.
2. Derrota (con `else if`): `fallos >= MAX_FALLOS`. Mostrad `caraDerrota`, enseñad la palabra entera y llamad también a `finalizarPartida()`.
3. Para enseñar la palabra con espacios: `palabraSecreta.split('').join(' ')`.



🔍 **Para entenderlo:** el árbitro que mira el marcador al final de cada jugada. No juega; solo decide si el partido sigue.



✅ **Comprobadlo:** ganad una partida (mirad la palabra en la consola si hace falta): mensaje de victoria y la caja queda desactivada. Después perded con `K W Q Z X Ñ`: muñeco completo, ojos en cruz, palabra destapada y caja desactivada. «Nueva Palabra» lo deja todo como al principio.



🆘 **Si no funciona:** si al perder sigue dejando escribir, falta `finalizarPartida()` o `juegoTerminado = true`. Si gana con la primera letra, estáis buscando `''` (vacío) en vez de `'_'`.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-7**. Intentadlo antes de mirar.

☐ ¡Hecho! Se puede ganar y perder



2

SOLUCIÓN 2 · CARPETA VERSION-2-TECLADO-EVENTOS

Teclado y eventos

Adiós a la caja de texto. Ahora el teclado de letras lo fabrica vuestro JavaScript, un único escuchador vigila las 27 teclas y, de regalo, también se puede jugar con el teclado del ordenador.

1 · Teclado | 2 · Iniciar | 3 · Casillas y vidas | 4 · Delegación | 5 · Aciertos y fallos | 6 · Teclado físico | 7 · Pista | 8 · Fin

SOLUCIÓN 2 · LO QUE CAMBIA RESPECTO A LA 1



Tres novedades antes de empezar

Palabras con ficha

```
{ palabra: 'CSS',
  categoria: 'Diseño Web',
  pista: 'Hojas de
estilo...' }
```

Cada palabra es un objeto con su categoría y su pista. La partida guarda el objeto entero en `partidaActual`.

Un `Set` para las usadas

```
letrasUsadas.add('A');
letrasUsadas.has('A'); //
true
letrasUsadas.clear();
```

Un `Set` no admite repetidos[3]. Se usa `add`, `has` y `clear` en vez de `push` e `includes`.

Casillas en vez de guiones

```
letrasAdivinadas =
  ['', 'S', ''];
```

Las letras sin descubrir son cadenas vacías `''`, no `'_'`. Cada letra se pinta en su propia cajita.

La validación de la Solución 1 (¿es una letra?, ¿está repetida?) ya viene escrita al principio de `jugarLetra()`. Y `mostrarMensaje()`, como antes, también.



Fabricar el teclado con JavaScript



🎯 **Objetivo:** crear un botón por cada letra del alfabeto (27, con la Ñ) dentro del contenedor `tecladoVirtual`.

📍 **Dónde:** bloque `TODO 3.1`, dentro de `renderizarTecladoVirtual()`.

Pistas, en orden

1. Recorred `LETRAS_ALFABETO` con `forEach`.
2. Para cada letra: `document.createElement('button')` [18].
3. Ponedle `type = 'button'`, `className = 'tecla'` y la letra en `textContent`.
4. Guardad la letra también en `btn.dataset.letra` [19]. Eso crea el atributo `data-letra="A"` en el HTML.
5. Metedlo en el contenedor con `tecladoVirtual.appendChild(btn)` [20].



✅ **Comprobadlo:** como nadie llama todavía a esta función, probadla a mano. Escribid en la consola `renderizarTecladoVirtual()`: aparecen las 27 teclas. Con `tecladoVirtual.children.length` debe salir `27`.



🔍 **Para entenderlo:** el `dataset` es la etiqueta de la taquilla del gimnasio. El botón puede mostrar lo que quiera por fuera, pero la etiqueta dice de quién es.



🆘 **Si no funciona:** si salen 27 botones sin texto, os falta el `textContent`. Si salen sin estilo, revisad que la clase es `'tecla'` sin punto delante: el punto solo se pone en los selectores.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-1**. Intentadlo antes de mirar.

☐ ¡Hecho! Salen las 27 teclas



Iniciar la partida



🎯 **Objetivo:** elegir una ficha al azar, preparar la memoria, esconder pista y muñeco, mostrar la categoría y fabricar el teclado.

📍 **Dónde:** bloques `TODO 4.1` a `4.6`, dentro de `iniciarJuego()`.

Pistas, en orden

1. `partidaActual` es `BANCO_PALABRAS` en una posición al azar (igual que en la Solución 1).
2. `letrasAdivinadas = Array(partidaActual.palabra.length).fill('')` [21]: un array del tamaño de la palabra, lleno de vacíos.
3. `letrasUsadas.clear()`, `fallos = 0`, `juegoTerminado = false`.
4. Añadid la clase `oculta` a `cajaPista`, activad `btnPista` y quitad `visible` a las piezas y a la cara.
5. `categoriaBadge.textContent = partidaActual.categoria` y llamad a `renderizarTecladoVirtual()`.
6. Terminad con `actualizarPalabraVisual()`, `actualizarMarcadorVidas()` y un mensaje.



✅ **Comprobadlo:** recargad. Ya se ve el teclado sin escribir nada en la consola, y la categoría aparece arriba. Escribid `partidaActual` en la consola: sale el objeto con sus tres datos. Las casillas todavía no se ven: llegan en el paso 3.



🆘 **Si no funciona:** si sale `Cannot read properties of null (reading 'palabra')`, estáis usando `partidaActual` antes de darle valor. El sorteo va lo primero.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-2**. Intentadlo antes de mirar.

☐ ¡Hecho! Teclado y categoría al cargar



Las casillas de la palabra y los corazones



🎯 **Objetivo:** una cajita por cada letra de la palabra (vacía o con la letra si ya se ha descubierto) y los corazones apagados según los fallos.

📍 **Dónde:** `TODO 6.1` en `actualizarPalabraVisual()` y `TODO 6.2` en `actualizarMarcadorVidas()`.

Pistas, en orden

1. La primera línea ya vacía el contenedor con `innerHTML = ''`[22]. Así no se acumulan casillas de una jugada a otra.
2. Un `for` sobre la palabra. En cada vuelta, un `span` con clase `casilla-letra`.
3. Si `letrasAdivinadas[i]` tiene algo, ponédselo como texto y añadid la clase `revelada`. Al final, `appendChild`.
4. Para los corazones: `vidasIconos.querySelectorAll('.corazon')` y, con `forEach((corazon, i) => ...)`, `classList.toggle('perdido', i >= vidas)`[2].



✅ **Comprobadlo:** recargad: tantas casillas vacías como letras tiene la palabra. En la consola, escribid `fallos = 2` y luego `actualizarMarcadorVidas()`: se apagan dos corazones y pone «(4 restantes)». Recargad para dejarlo bien.



🔍 **Para entenderlo:** `toggle` con segundo parámetro es un interruptor con orden: «enciende si se cumple, apaga si no». Os ahorra el `if / else`.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-3**. Intentadlo antes de mirar.

☐ ¡Hecho! Casillas y corazones correctos



Un solo escuchador para 27 teclas



🎯 **Objetivo:** que al pulsar cualquier tecla en pantalla se llame a `jugarLetra()` con su letra, y que esa tecla quede desactivada.

📍 **Dónde:** `TODO 8.1` (al final del archivo) y `TODO 5.1` y `5.2` en `jugarLetra()`.

Pistas, en orden

1. Un único `addEventListener('click', ...)` en `tecladoVirtual`, no en cada tecla.
2. Dentro, `evento.target` [23] es lo que se ha pulsado. `evento.target.closest('.tecla')` [24] sube hasta encontrar la tecla (o devuelve `null` si se pulsó en un hueco).
3. Si hay tecla y no está desactivada, `jugarLetra(tecla.dataset.letra)`.
4. En `jugarLetra()`: `letrasUsadas.add(letra)`, buscad la tecla con `querySelector(`[data-letra="${letra}"]`)` [25] y ponedle `disabled = true`.



🔍 **Para entenderlo:** el conserje de un edificio. En vez de poner un portero en cada piso, uno solo en la entrada pregunta «¿a qué piso va?». El clic sube desde la tecla hasta el contenedor por el burbujeo[26], y allí lo atiende el conserje.



✅ **Comprobadlo:** pulsad varias teclas: cada una se queda apagada y no se puede volver a pulsar. Escribid `letrasUsadas` en la consola: aparecen las que habéis pulsado, sin repetir.



🆘 **Si no funciona:** si no pasa nada, poned un `console.log(evento.target)` dentro del escuchador. Si en la consola tampoco sale nada, el escuchador no está en `tecladoVirtual`. Si sale pero no juega, revisad que leéis `dataset.letra`, no `dataset.Letra`.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-4**. Intentadlo antes de mirar.

☐ ¡Hecho! Las teclas se apagan al pulsarlas



Aciertos en verde, fallos en rojo



🎯 **Objetivo:** si la letra está, destaparla en las casillas y pintar la tecla de acierto; si no, pintarla de fallo, sumar el fallo y dibujar la pieza.

📍 **Dónde:** bloques `TODO 5.3` y `5.4`, en `jugarLetra()`, después de desactivar la tecla.

Pistas, en orden

1. `if (partidaActual.palabra.includes(letra))`.
2. Acierto: `boton.classList.add('acierto')` y el mismo bucle de la Solución 1 para rellenar `letrasAdivinadas[i]`.
3. Fallo: `boton.classList.add('fallo')`, `fallos++` y `partesAhorcado[fallos - 1].classList.add('visible')`.
4. Fuera del `if`: `actualizarPalabraVisual()`, `actualizarMarcadorVidas()` y `verificarFinJuego()`.



✅ **Comprobadlo:** pulsad una letra que esté (mirad `partidaActual.palabra`): se destapa en su casilla y la tecla queda verde. Pulsad `K`: tecla roja, cabeza dibujada y un corazón menos.



🆘 **Si no funciona:** si la casilla no cambia aunque la tecla se ponga verde, os falta llamar a `actualizarPalabraVisual()` después del bucle. Pintar en memoria no es pintar en pantalla.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-5**. Intentadlo antes de mirar.

☐ ¡Hecho! Verde, rojo y muñeco



Jugar también con el teclado del ordenador



🎯 **Objetivo:** que pulsar una letra en el teclado físico haga exactamente lo mismo que pulsarla en pantalla.

📍 **Dónde:** bloque `TODO 8.2`, al final del archivo.

Pistas, en orden

1. Escuchad `'keydown'` [13] en `window`, así funciona esté donde esté el foco.
2. Si se pulsa con `Ctrl`, `Alt` o `Meta` (`evento.ctrlKey`…), `return`: no queréis que `Ctrl`+`R` cuente como una R.
3. `evento.key` [14] trae la tecla; pasadla a mayúscula.
4. Si está en `LETRAS_ALFABETO`, llamad a `jugarLetra(letra)`. La misma función que usa el ratón.



🔍 **Para entenderlo:** dos puertas que dan al mismo pasillo. Da igual por cuál entréis: acabáis en `jugarLetra()`.



✅ **Comprobadlo:** pulsad letras en el teclado del ordenador: se juegan y su tecla en pantalla se apaga. Pulsad `F5`, `Enter` o un número: no pasa nada en el juego.



🆘 **Si no funciona:** si las letras se juegan dos veces o la tecla se pone verde y roja a la vez, revisad que la validación del principio de `jugarLetra()` sigue ahí: es la que impide repetir.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-6**. Intentadlo antes de mirar.

☐ ¡Hecho! Funciona con el teclado físico



El botón de pista



🎯 **Objetivo:** al pulsar «Pedir Pista», mostrar la pista de la palabra y desactivar el botón para que solo se pueda pedir una vez.

📍 **Dónde:** bloque `TODO 6.3`, dentro de `alternarPista()`. El escuchador del botón ya está escrito.

Pistas, en orden

1. Si la partida ha terminado, `return`.
2. `textoPista.textContent = partidaActual.pista`.
3. Quitad la clase `oculta` a `cajaPista`.
4. `btnPista.disabled = true`.



✅ **Comprobadlo:** pulsad «Pedir Pista»: aparece el texto y el botón se apaga. Pulsad «Nueva Palabra»: la pista vuelve a esconderse y el botón se enciende (eso lo hicisteis en el paso 2).



🆘 **Si no funciona:** si la pista es la de la palabra anterior, estáis escribiendo el texto en `iniciarJuego()` antes del sorteo. Escribidlo aquí, al pedirla.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-7**. Intentadlo antes de mirar.

☐ ¡Hecho! La pista aparece una vez



Fin de partida: bloquear el teclado



🎯 **Objetivo:** detectar victoria (no queda ninguna casilla vacía) o derrota (vidas agotadas) y, en los dos casos, apagar todas las teclas y la pista.

📍 **Dónde:** `TODO 7.1` y `7.2` en `verificarFinJuego()`, más una función nueva de ayuda.

Pistas, en orden

1. Victoria: `!letrasAdivinadas.includes('')`. Ojo: ahora se busca la cadena vacía, no el guion.
2. Derrota: `fallos >= MAX_FALLOS`. Mostrad la cara y escribid cada letra de la palabra en su casilla.
3. Como bloquear el teclado se hace en los dos casos, cread una función `bloquearTeclado()` que recorra todas las `.tecla` y las desactive, y que desactive también `btnPista`.



✅ **Comprobadlo:** ganad una partida: mensaje de victoria y teclado apagado entero. Perded con `K W Q Z X Ñ`: muñeco completo, ojos en cruz y la palabra escrita en las casillas. Pulsar letras ya no hace nada.



🆘 **Si no funciona:** si os sale «Victoria» nada más empezar, estáis buscando `'_'`, que ya no existe en esta solución. Copiar y pegar de la Solución 1 tiene estas cosas.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-8**. Intentadlo antes de mirar.

☐ ¡Hecho! Se gana, se pierde y se bloquea



3

SOLUCIÓN 3 · CARPETA VERSION-3-AVANZADA-POO

Avanzada con clases

El juego se reparte entre cuatro clases. Tres ya vienen escritas: el sonido, las estadísticas y el motor que sabe las reglas. Vosotros programáis la cuarta, `AhorcadoUI`, la que conecta el motor con la pantalla.

Las 3 clases · 1 · Caché del DOM · 2 · Iniciar · 3 · Teclado y salud · 4 · Jugar · 5 · Teclado físico · 6 · Fin · 7 · Botones · 8 · Temporizador

SOLUCIÓN 3 · PREPARATIVOS (YA ESCRITOS)



Tres clases que ya funcionan

CLASE	QUÉ HACE
`SoundSynthesizer`	Genera pitidos con la Web Audio API[27]: `playSuccess()`, `playError()` y `toggleMute()`.
`StatsRepository`	Guarda partidas, victorias y rachas en `localStorage`[28], convertidas a texto con `JSON`[29]. Su método clave: `recordGame(victoria)`.
`AhorcadoEngine`	Las reglas del juego, sin tocar la pantalla. `iniciarNuevaPartida()` y `probarLetra(letra)`.



🔍 **Para entenderlo:** el motor es el árbitro con el reglamento; la interfaz es el marcador del estadio. El árbitro no enciende bombillas y el marcador no decide si fue gol.

Lo más importante es lo que devuelve `probarLetra()`: un objeto con todo lo que la pantalla necesita saber.

```
{ letra: 'K', acierto: false, fallos: 1, maxFallos: 6,
  victoria: false, derrota: false, palabraSecreta: 'JAVASCRIPT' }
```



✅ **Comprobadlo:** las clases se pueden probar solas en la consola, sin pantalla:
`const m = new AhorcadoEngine();`
`m.iniciarNuevaPartida();`
`m.probarLetra('A');`
Mirad el objeto que devuelve. Probad otra vez `'A'`: devuelve `null` porque está repetida.



Guardar los elementos en `this.dom`



🎯 **Objetivo:** en el constructor, buscar una sola vez todos los elementos de la página y guardarlos en un objeto `this.dom`.

📍 **Dónde:** dentro del `constructor()` de `AhorcadoUI`, debajo de `this.statsRepo`, donde pone «Cache de elementos».

Pistas, en orden

1. Cread `this.dom = { ... }`. Cada propiedad es un `document.getElementById(...)`. Los `id` los tenéis en el `index.html`.
2. Necesitáis al menos: `teclado-virtual`, `palabra-display`, `categoria-display`, `vidas-numero`, `barra-salud-fill`, `mensaje-estado`, los de la pista, el selector de dificultad, el botón de audio, el de nueva partida y los del modal de fin.
3. Las 8 piezas del muñeco (aquí hay manos) en un array: `['cabeza', 'cuerpo', ...].map(n => document.getElementById('parte-' + n))`.
4. De momento, al final del constructor, un `console.log(this.dom)`.



🔍 **Para entenderlo:** es la agenda de teléfonos. Buscáis cada número una vez y lo apuntáis; luego llamáis sin volver a mirar la guía.



✅ **Comprobadlo:** recargad y abrid el objeto que sale en la consola. Ninguna propiedad puede valer `null`. Si alguna lo es, ese `id` está mal escrito.



🆘 **Si no funciona:** dentro de una clase, todo lo que sea «del objeto» lleva `this.` delante[30]. Escribir solo `dom.teclado` da `ReferenceError: dom is not defined`.



¿Atascados? La solución de este paso está en el anexo, apartado **S3-1**. Intentadlo antes de mirar.

☐ ¡Hecho! Ningún null en this.dom



Iniciar la partida y pintar las casillas



🎯 **Objetivo:** un método `iniciar()` que pide partida al motor y prepara la pantalla, y otro `renderPalabra()` que dibuja las casillas.

📍 **Dónde:** dos métodos nuevos dentro de la clase, debajo del constructor. Al final del constructor, cambiad el `console.log` por `this.iniciar();`.

Pistas, en orden

1. `const item = this.engine.iniciarNuevaPartida();` devuelve la ficha con `categoria` y `pista`.
2. Esconded el modal (`oculta`), las piezas y la cara (quitad `visible`).
3. Escribid categoría y pista, ocultad `cajaPista` y activad `btnPista`.
4. `renderPalabra()` vacía `this.dom.palabra` y recorre `this.engine.letrasAdivinadas` : un `span.casilla-letra` por letra, con la clase `revelada` si no está vacía.



✅ **Comprobadlo:** recargad: aparecen la categoría arriba y las casillas vacías de la palabra. El teclado sigue vacío; llega en el paso 3.



🆘 **Si no funciona:** si sale `this.renderPalabra is not a function` , el método está fuera de la llave de cierre de la clase o tiene la palabra `function` delante. Dentro de una clase, los métodos se escriben sin `function` [4].



¿Atascados? La solución de este paso está en el anexo, apartado **S3-2**. Intentadlo antes de mirar.

☐ ¡Hecho! Categoría y casillas al cargar



El teclado y la barra de salud



🎯 **Objetivo:** fabricar las 27 teclas y mostrar las vidas como texto («6 / 6») y como barra.

📍 **Dónde:** dos métodos nuevos, `renderTeclado()` y `actualizarSalud()`, y sus llamadas al final de `iniciar()`.

Pistas, en orden

1. `renderTeclado()` es la función del paso 1 de la Solución 2, con `this.dom.teclado` y `ALFABETO`.
2. En `actualizarSalud()`, el máximo depende de la dificultad: `this.engine.configDificultades[this.engine.dificultad].maxFallos`.
3. Vidas = máximo − `this.engine.fallos`. El texto: `` `${vidas} / ${max}` ``.
4. La barra: `this.dom.saludFill.style.width = `${(vidas / max) * 100}%` `.



✅ **Comprobadlo:** recargad: teclado completo, «6 / 6» y barra llena. Todavía no se puede jugar, pero ya tiene toda la pinta.



🆘 **Si no funciona:** si la barra no se mueve nunca, os falta el `%` al final. Un `width` sin unidades no vale en CSS.



¿Atascados? La solución de este paso está en el anexo, apartado **S3-3**. Intentadlo antes de mirar.

☐ ¡Hecho! Teclado, «6 / 6» y barra llena



Jugar una letra: el motor decide, la interfaz pinta



🎯 **Objetivo:** al pulsar una tecla, preguntar al motor y, según su respuesta, pintar el acierto o el fallo con su sonido.

📍 **Dónde:** un método `bindEvents()` con el escuchador del teclado, otro `jugar(letra)`, y la llamada `this.bindEvents();` en el constructor, antes de `this.iniciar();`.

Pistas, en orden

1. En `bindEvents()`: la misma delegación de la Solución 2, pero llamando a `this.jugar(...)`. Usad funciones flecha[31]: así `this` sigue siendo la interfaz dentro del escuchador.
2. En `jugar(letra)`: `const res = this.engine.probarLetra(letra);`. Si `res` es `null`, `return`.
3. Desactivad la tecla y añadidle `'acierto'` o `'fallo'` según `res.acierto`.
4. Acierto: `this.audio.playSuccess()` y `this.renderPalabra()`. Fallo: `playError()`, encended `partesSvg[res.fallos - 1]` y `this.actualizarSalud()`.



✅ **Comprobadlo:** pulsad letras: los aciertos suenan agudo y se destapan; los fallos suenan grave, dibujan pieza y bajan la barra. Fijaos en que `jugar()` no tiene ni un solo `includes`: las reglas son cosa del motor.



🆘 **Si no funciona:** si sale `Cannot read properties of undefined (reading 'probarLetra')`, habéis escrito el escuchador con `function (evento) {...}`. Con `function`, `this` deja de ser la interfaz. Cambiadla por una flecha.



¿Atascados? La solución de este paso está en el anexo, apartado **S3-4**. Intentadlo antes de mirar.

☐ ¡Hecho! Se juega con el ratón y suena



El teclado del ordenador, otra vez



🎯 **Objetivo:** que las letras del teclado físico también llamen a `this.jugar()`.

📍 **Dónde:** dentro de `bindEvents()`, debajo del escuchador del clic.

Pistas, en orden

1. `window.addEventListener('keydown', evento => { ... })`, con flecha.
2. Ignorad las combinaciones con `Ctrl`, `Alt` o `Meta`.
3. Pasad `evento.key` a mayúscula y, si está en `ALFABETO`, `this.jugar(letra)`.

Es la misma idea de la Solución 2. La diferencia está en a quién se llama: a un método del objeto, no a una función suelta.



✅ **Comprobadlo:** jugad una partida entera solo con el teclado físico. Las teclas de la pantalla se van apagando igual que con el ratón.



🆘 **Si no funciona:** si una letra repetida no hace nada, está bien: el motor devuelve `null` y `jugar()` sale por el `return`. Es el comportamiento que queremos.



¿Atascados? La solución de este paso está en el anexo, apartado **S3-5**. Intentadlo antes de mirar.

☐ ¡Hecho! Se juega con el teclado físico



Fin de partida con ventana y estadísticas



🎯 **Objetivo:** cuando el motor diga victoria o derrota, bloquear el teclado, guardar la partida en las estadísticas y abrir la ventana de fin.

📍 **Dónde:** un método nuevo `finPartida(res)` y, al final de `jugar()`, la línea `if (res.victoria || res.derrota) this.finPartida(res);`.

Pistas, en orden

1. Desactivad todas las `.tecla` y `btnPista`.
2. `this.statsRepo.recordGame(res.victoria)`. Una línea, y la partida queda guardada en `localStorage`.
3. Victoria: icono 🏆, título y descripción. Derrota: icono 💀, cara de derrota y la palabra escrita en las casillas con `res.palabraSecreta`.
4. Rellenad la racha (`this.statsRepo.stats.rachaActual`) y las vidas que quedaban.
5. Quitad `oculta` al modal dentro de un `setTimeout` de unos 400 ms, para que dé tiempo a ver la última jugada.



✅ **Comprobadlo:** ganad y perded una partida: sale la ventana con el resultado. Recargad la página y escribid en la consola `localStorage.getItem('ahorcado_stats_pro_2026')`: siguen ahí vuestras partidas, aunque hayáis recargado.



🔍 **Para entenderlo:** `localStorage` es una libreta que el navegador guarda en un cajón. Al cerrar la página, la libreta sigue en el cajón; la próxima vez la volvéis a abrir.



¿Atascados? La solución de este paso está en el anexo, apartado **S3-6**. Intentadlo antes de mirar.

☐ ¡Hecho! Ventana de fin y estadísticas guardadas



Dificultad, sonido, pista y nueva partida



🎯 **Objetivo:** que funcionen el selector de dificultad, el botón de sonido, la pista, «Nueva partida» y «Jugar otra vez».

📍 **Dónde:** dentro de `bindEvents()`, debajo del teclado físico.

Pistas, en orden

1. Selector: evento `'change'` [32]. Guardad `evento.target.value` en `this.engine.dificultad` y llamad a `this.iniciar()`.
2. Sonido: `this.audio.toggleMute()` devuelve si queda silenciado. Cambiad el icono entre 🔇 y 🔊.
3. Pista: quitad `oculta` a la caja y desactivad el botón.
4. «Nueva partida» y «Jugar otra vez»: los dos llaman a `this.iniciar()` (que ya esconde el modal).



✅ **Comprobadlo:** elegid «Fácil»: el marcador pasa a «8 / 8» y empieza otra partida. Con 8 fallos aparecen también las manos. Elegid «Difícil»: «5 / 5». Silenciad el sonido, recargad y comprobad en la consola que `localStorage.getItem('ahorcado_muted')` vale `'true'`.



⚠️ **Ojo:** el modo Difícil ya cambia las vidas, pero todavía no tiene reloj. Los 45 segundos llegan en el paso 8.



¿Atascados? La solución de este paso está en el anexo, apartado **S3-7**. Intentadlo antes de mirar.

☐ ¡Hecho! Todos los botones funcionan



45 segundos de reloj en el modo Difícil



🎯 **Objetivo:** en Difícil, mostrar la barra de tiempo, descontar un segundo cada segundo y dar la partida por perdida al llegar a cero. En Fácil y Normal, la barra no aparece.

📍 **Dónde:** dos retoques en `AhorcadoEngine` y un método nuevo en `AhorcadoUI`. La barra ya está en el HTML: `caja-temporizador`, `tiempo-texto` y `barra-tiempo-fill`.

Pistas, en orden

1. **Motor.** Añadid `tiempoMax: 45` a la configuración de `dificil`. Así el tiempo es una regla del juego, como las vidas.
2. **Motor.** Un método `tiempoAgotado()` que ponga `juegoTerminado = true` y devuelva un informe igual que `probarLetra()`, con `derrota: true` y `letra: null`.
3. **Interfaz.** Guardad en `this.dom` los tres elementos de la barra y `modal-stat-tiempo`. En el constructor, `this.temporizador = null;`.
4. En `iniciarTemporizador()`: parad el reloj anterior con `clearInterval` [33]. Si la dificultad no tiene `tiempoMax`, ocultad la caja y `return`.
5. Si lo tiene, con `setInterval(..., 1000)` [34] restad un segundo, repintad texto y barra y, al llegar a 0, `this.finPartida(this.engine.tiempoAgotado())`.
6. Llamadlo al final de `iniciar()`, apuntando la hora de salida con `Date.now()` [35]. En `finPartida()`, lo primero, `clearInterval(this.temporizador)`.



🔍 **Para entenderlo:** el reloj de ajedrez. Se pone en marcha al empezar, se para en cuanto termina la partida y, si llega a cero, pierdes aunque el tablero vaya bien.



✅ **Comprobadlo:** elegid «Difícil»: aparece la barra con `45s` y baja cada segundo. Dejad que llegue a cero sin jugar: teclado apagado, palabra destapada y la ventana dice «¡Tiempo agotado!». Ganad otra partida a tiempo: el reloj se queda quieto. Volved a «Normal»: la barra desaparece.



🆘 **Si no funciona:** si el reloj corre al doble de velocidad después de cambiar de partida, os falta el `clearInterval` del principio: hay dos relojes funcionando a la vez. Si al ganar aparece después «¡Tiempo agotado!», falta pararlo en `finPartida()`.



¿Atascados? La solución de este paso está en el anexo, apartado **S3-8**. Intentadlo antes de mirar.

☐ ¡Hecho! El reloj funciona en Difícil

COMPROBACIÓN FINAL · PARA LAS TRES SOLUCIONES



Antes de entregar, pasad esta lista

HITO	QUÉ HACÉIS	QUÉ TIENE QUE PASAR	✓
1	Recargad la página	Tantos huecos como letras y todas las vidas	☐
1	Pulsad «Nueva palabra» tres veces	La palabra (y su longitud) va cambiando	☐
2	Probad `7` en la Solución 1, o una letra repetida en cualquiera	Aviso o nada, y las vidas no bajan	☐
3	Una letra que se repita en la palabra	Se destapan todas sus apariciones	☐
4	`K`, `W`, `Q`…	Cabeza, tronco, brazos y piernas, en ese orden	☐
5	Descubrid la palabra entera	Mensaje de victoria y ya no se puede jugar	☐
6	`K W Q Z X Ñ`	Muñeco completo, ojos en cruz y palabra destapada	☐
—	Solución 3 en Difícil: esperad sin jugar	El reloj baja de 45 a 0 y la partida termina sola	☐
—	Partida nueva después de perder	Todo limpio: sin piezas, sin letras, vidas completas	☐
—	Consola abierta durante todas las pruebas	Ningún error en rojo	☐

Y quitad los `console.log` provisionales. Funcionan, sí, pero ensucian la entrega.

SI ALGO FALLA · TRADUCCIÓN DE LA CONSOLA



Lo que os dice la consola y lo que significa

MENSAJE EN ROJO	QUÉ SIGNIFICA	DÓNDE MIRAR
`Uncaught ReferenceError: palabraSecretta is not defined`	Usáis un nombre que no existe. Casi siempre es una errata.	Copiad el nombre desde donde se declara, no lo escribáis de memoria.
`Uncaught SyntaxError: Unexpected token '}'`	Sobra o falta una llave, un paréntesis o una coma.	La línea que indica la consola o la de justo encima.
`Cannot read properties of null (reading 'classList')`	Un `getElementById` no ha encontrado nada.	El `id`: comparadlo letra a letra con el `index.html`.
`Cannot read properties of undefined (reading 'classList')`	Pedís una posición del array que no existe, casi siempre `partesAhorcado[6]`.	El `- 1` del índice, o falta el `if (parte)`.
`this.jugar is not a function`	Dentro de un escuchador escrito con `function`, `this` ya no es vuestro objeto.	Usad funciones flecha en los escuchadores de la Solución 3.
`Assignment to constant variable`	Intentáis cambiar una variable declarada con `const`.	`fallos`, `palabraSecreta`… van con `let`.
Sin error, pero no cambia nada	No habéis guardado o no habéis recargado. Lo de siempre.	`Ctrl` + `S` y `F5`.

Un consejo que vale para toda vuestra vida programando: leed el mensaje de error entero. Os dice qué ha pasado y en qué línea. Ignorarlo es como ir al médico y taparse los oídos.



Solución 1 · Básica lineal

Solo para comprobar, no para copiar. Si miráis aquí antes de intentarlo, os perdéis la parte en la que de verdad se aprende.

S1-1 · Solución 1 · Paso 1 de 7 · Hito 1 — Elegir la palabra y taparla con guiones

```
// TODO 3.1
const indice = Math.floor(Math.random() * PALABRAS.length);
palabraSecreta = PALABRAS[indice];

// TODO 3.2
letrasAdivinadas = [];
for (let i = 0; i < palabraSecreta.length; i++) {
  letrasAdivinadas.push('_');
}

// TODO 3.3
letrasUsadas = [];
fallos = 0;
juegoTerminado = false;
```

S1-2 · Solución 1 · Paso 2 de 7 · Hito 1 — Pintar el tablero en pantalla

```
// Dentro de actualizarPantalla()
palabraDisplay.textContent = letrasAdivinadas.join(' ');
vidasRestantes.textContent = MAX_FALLOS - fallos;
letrasUsadasDisplay.textContent = letrasUsadas.length > 0 ? letrasUsadas.join(', ') : 'Ninguna aún';

// Al final de iniciarJuego()
partesAhorcado.forEach(parte => parte.classList.remove('visible'));
caraDerrota.classList.remove('visible');

inputLetra.disabled = false;
btnProbar.disabled = false;
inputLetra.value = '';
inputLetra.focus();

actualizarPantalla();
mostrarMensaje('¡Nueva partida! Escribe una letra y pulsa Enter.');
```

S1-3 · Solución 1 · Paso 3 de 7 · Escuchadores — Que el botón y la tecla Enter hagan algo

```
// TODO 7.1
btnProbar.addEventListener('click', procesarLetra);

// TODO 7.2
inputLetra.addEventListener('keydown', evento => {
  if (evento.key === 'Enter') procesarLetra();
});

// TODO 7.3
btnReiniciar.addEventListener('click', iniciarJuego);
```

S1-4 · Solución 1 · Paso 4 de 7 · Hito 2 — Validar la letra antes de jugarla

```
// TODO 4.2
if (!/^[A-ZÑ]$/.test(letra)) {
  mostrarMensaje('Introduce una letra válida (A-Z).', 'aviso');
  return;
}
if (letrasUsadas.includes(letra)) {
  mostrarMensaje(`La letra "${letra}" ya está probada.`, 'aviso');
  return;
}

// TODO 4.3
letrasUsadas.push(letra);

// TODO 4.5 (al final de procesarLetra)
actualizarPantalla();
verificarFinJuego();
```

S1-5 · Solución 1 · Paso 5 de 7 · Hito 3 — Destapar las letras acertadas

```
// TODO 4.4 (parte del acierto)
if (palabraSecreta.includes(letra)) {
  for (let i = 0; i < palabraSecreta.length; i++) {
    if (palabraSecreta[i] === letra) {
      letrasAdivinadas[i] = letra;
    }
  }
  mostrarMensaje(`¡Acierto! La letra "${letra}" está en la palabra.`, 'exito');
} else {
  // Paso 6
}
```

S1-6 · Solución 1 · Paso 6 de 7 · Hito 4 — Contar el fallo y dibujar una pieza

```
// TODO 4.4 (parte del fallo)
} else {
  fallos++;
  dibujarSiguienteParte(fallos);
  mostrarMensaje(`¡Fallo! La letra "${letra}" no está.`, 'error');
}

// TODO 5.4
function dibujarSiguienteParte(numeroFallo) {
  const parte = partesAhorcado[numeroFallo - 1];
  if (parte) parte.classList.add('visible');
}
```

S1-7 · Solución 1 · Paso 7 de 7 · Hitos 5 y 6 — ¿Has ganado o has perdido?

```
function verificarFinJuego() {
  if (!letrasAdivinadas.includes('_')) {
    juegoTerminado = true;
    mostrarMensaje(`🎉 ¡Has ganado! La palabra es "${palabraSecreta}".`, 'exito');
    finalizarPartida();
  } else if (fallos >= MAX_FALLOS) {
    juegoTerminado = true;
    caraDerrota.classList.add('visible');
    palabraDisplay.textContent = palabraSecreta.split('').join(' ');
    mostrarMensaje(`💀 ¡Has perdido! La palabra era "${palabraSecreta}".`, 'error');
    finalizarPartida();
  }
}
```



Solución 2 · Teclado y eventos

Solo para comprobar, no para copiar. Si miráis aquí antes de intentarlo, os perdéis la parte en la que de verdad se aprende.

S2-1 · Solución 2 · Paso 1 de 8 · Preparativos — Fabricar el teclado con JavaScript

```
LETRAS_ALFABETO.forEach(letra => {
  const btn = document.createElement('button');
  btn.type = 'button';
  btn.className = 'tecla';
  btn.textContent = letra;
  btn.dataset.letra = letra;
  tecladoVirtual.appendChild(btn);
});
```

S2-2 · Solución 2 · Paso 2 de 8 · Hito 1 — Iniciar la partida

```
const indice = Math.floor(Math.random() * BANCO_PALABRAS.length);
partidaActual = BANCO_PALABRAS[indice];

letrasAdivinadas = Array(partidaActual.palabra.length).fill('');
letrasUsadas.clear();
fallos = 0;
juegoTerminado = false;

cajaPista.classList.add('oculta');
btnPista.disabled = false;
partesAhorcado.forEach(parte => parte.classList.remove('visible'));
caraDerrota.classList.remove('visible');

categoriaBadge.textContent = partidaActual.categoria;
renderizarTecladoVirtual();

actualizarPalabraVisual();
actualizarMarcadorVidas();
mostrarMensaje('¡Partida lista! Elige una letra.');
```

S2-3 · Solución 2 · Paso 3 de 8 · Hito 1 — Las casillas de la palabra y los corazones

```
// TODO 6.1
for (let i = 0; i < partidaActual.palabra.length; i++) {
  const casilla = document.createElement('span');
  casilla.className = 'casilla-letra';
  if (letrasAdivinadas[i]) {
    casilla.textContent = letrasAdivinadas[i];
    casilla.classList.add('revelada');
  }
  palabraDisplay.appendChild(casilla);
}

// TODO 6.2
const corazones = vidasIconos.querySelectorAll('.corazon');
corazones.forEach((corazon, i) => corazon.classList.toggle('perdido', i >= vidas));
```

S2-4 · Solución 2 · Paso 4 de 8 · Delegación de eventos — Un solo escuchador para 27 teclas

```
// TODO 8.1
tecladoVirtual.addEventListener('click', evento => {
  const tecla = evento.target.closest('.tecla');
  if (tecla && !tecla.disabled) jugarLetra(tecla.dataset.letra);
});

// TODO 5.1 y 5.2, dentro de jugarLetra()
letrasUsadas.add(letra);

const boton = tecladoVirtual.querySelector(`[data-letra="${letra}"]`);
if (boton) boton.disabled = true;
```

S2-5 · Solución 2 · Paso 5 de 8 · Hitos 3 y 4 — Aciertos en verde, fallos en rojo

```
if (partidaActual.palabra.includes(letra)) {
  if (boton) boton.classList.add('acierto');
  for (let i = 0; i < partidaActual.palabra.length; i++) {
    if (partidaActual.palabra[i] === letra) letrasAdivinadas[i] = letra;
  }
  mostrarMensaje(`¡Acierto con "${letra}"!`, 'exito');
} else {
  if (boton) boton.classList.add('fallo');
  fallos++;
  if (partesAhorcado[fallos - 1]) partesAhorcado[fallos - 1].classList.add('visible');
  mostrarMensaje(`¡Fallo con "${letra}"!`, 'error');
}

actualizarPalabraVisual();
actualizarMarcadorVidas();
verificarFinJuego();
```

S2-6 · Solución 2 · Paso 6 de 8 · Teclado físico — Jugar también con el teclado del ordenador

```
window.addEventListener('keydown', evento => {
  if (evento.ctrlKey || evento.altKey || evento.metaKey) return;
  const letra = evento.key.toUpperCase();
  if (LETRAS_ALFABETO.includes(letra)) jugarLetra(letra);
});
```

S2-7 · Solución 2 · Paso 7 de 8 · Pista — El botón de pista

```
function alternarPista() {
  if (juegoTerminado) return;
  textoPista.textContent = partidaActual.pista;
  cajaPista.classList.remove('oculta');
  btnPista.disabled = true;
}
```

S2-8 · Solución 2 · Paso 8 de 8 · Hitos 5 y 6 — Fin de partida: bloquear el teclado

```
function verificarFinJuego() {
  if (!letrasAdivinadas.includes('')) {
    juegoTerminado = true;
    bloquearTeclado();
    mostrarMensaje(`🏆 ¡Victoria! Has descubierto "${partidaActual.palabra}".`, 'exito');
  } else if (fallos >= MAX_FALLOS) {
    juegoTerminado = true;
    caraDerrota.classList.add('visible');
    bloquearTeclado();
    const casillas = palabraDisplay.querySelectorAll('.casilla-letra');
    casillas.forEach((casilla, i) => { casilla.textContent = partidaActual.palabra[i]; });
    mostrarMensaje(`💀 ¡Derrota! La palabra era "${partidaActual.palabra}".`, 'error');
  }
}

function bloquearTeclado() {
  tecladoVirtual.querySelectorAll('.tecla').forEach(tecla => tecla.disabled = true);
  btnPista.disabled = true;
}
```



Solución 3 · Avanzada con clases

Solo para comprobar, no para copiar. Si miráis aquí antes de intentarlo, os perdéis la parte en la que de verdad se aprende.

S3-1 · Solución 3 · Paso 1 de 8 · Preparativos — Guardar los elementos en this.dom

```
this.dom = {
  teclado: document.getElementById('teclado-virtual'),
  palabra: document.getElementById('palabra-display'),
  categoria: document.getElementById('categoria-display'),
  vidas: document.getElementById('vidas-numero'),
  saludFill: document.getElementById('barra-salud-fill'),
  mensaje: document.getElementById('mensaje-estado'),
  btnPista: document.getElementById('btn-pista'),
  cajaPista: document.getElementById('caja-pista'),
  textoPista: document.getElementById('texto-pista'),
  selectDif: document.getElementById('select-dificultad'),
  btnAudio: document.getElementById('btn-audio'),
  iconoAudio: document.getElementById('icono-audio'),
  btnNueva: document.getElementById('btn-nueva-partida'),
  modalFin: document.getElementById('modal-fin'),
  modalIcono: document.getElementById('modal-icono'),
  modalTitulo: document.getElementById('modal-titulo'),
  modalDesc: document.getElementById('modal-descripcion'),
  modalRacha: document.getElementById('modal-stat-racha'),
  modalVidas: document.getElementById('modal-stat-vidas'),
  modalBtn: document.getElementById('modal-btn-jugar'),
  caraDerrota: document.getElementById('parte-cara-derrota'),
  partesSvg: ['cabeza', 'cuerpo', 'brazo-izq', 'brazo-der',
              'pierna-izq', 'pierna-der', 'mano-izq', 'mano-der']
    .map(nombre => document.getElementById('parte-' + nombre))
};
console.log(this.dom);
```

S3-2 · Solución 3 · Paso 2 de 8 · Hito 1 — Iniciar la partida y pintar las casillas

```
iniciar() {
  const item = this.engine.iniciarNuevaPartida();

  this.dom.modalFin.classList.add('oculta');
  this.dom.partesSvg.forEach(parte => parte.classList.remove('visible'));
  this.dom.caraDerrota.classList.remove('visible');

  this.dom.categoria.textContent = item.categoria;
  this.dom.textoPista.textContent = item.pista;
  this.dom.cajaPista.classList.add('oculta');
  this.dom.btnPista.disabled = false;
  this.dom.mensaje.textContent = '¡Partida lista! Elige una letra.';

  this.renderPalabra();
}

renderPalabra() {
  this.dom.palabra.innerHTML = '';
  this.engine.letrasAdivinadas.forEach(letra => {
    const casilla = document.createElement('span');
    casilla.className = 'casilla-letra';
    if (letra) {
      casilla.textContent = letra;
      casilla.classList.add('revelada');
    }
    this.dom.palabra.appendChild(casilla);
  });
}
```

S3-3 · Solución 3 · Paso 3 de 8 · Hito 1 — El teclado y la barra de salud

```
// Al final de iniciar()
this.renderTeclado();
this.actualizarSalud();

renderTeclado() {
  this.dom.teclado.innerHTML = '';
  ALFABETO.forEach(letra => {
    const tecla = document.createElement('button');
    tecla.type = 'button';
    tecla.className = 'tecla';
    tecla.textContent = letra;
    tecla.dataset.letra = letra;
    this.dom.teclado.appendChild(tecla);
  });
}

actualizarSalud() {
  const max = this.engine.configDificultades[this.engine.dificultad].maxFallos;
  const vidas = max - this.engine.fallos;
  this.dom.vidas.textContent = `${vidas} / ${max}`;
  this.dom.saludFill.style.width = `${(vidas / max) * 100}%`;
}
```

S3-4 · Solución 3 · Paso 4 de 8 · Hitos 3 y 4 — Jugar una letra: el motor decide, la interfaz pinta

```
// En el constructor, antes de this.iniciar():
this.bindEvents();

bindEvents() {
  this.dom.teclado.addEventListener('click', evento => {
    const tecla = evento.target.closest('.tecla');
    if (tecla && !tecla.disabled) this.jugar(tecla.dataset.letra);
  });
}

jugar(letra) {
  const res = this.engine.probarLetra(letra);
  if (!res) return;

  const tecla = this.dom.teclado.querySelector(`[data-letra="${letra}"]`);
  if (tecla) {
    tecla.disabled = true;
    tecla.classList.add(res.acierto ? 'acierto' : 'fallo');
  }

  if (res.acierto) {
    this.audio.playSuccess();
    this.renderPalabra();
  } else {
    this.audio.playError();
    const parte = this.dom.partesSvg[res.fallos - 1];
    if (parte) parte.classList.add('visible');
    this.actualizarSalud();
  }
}
```

S3-5 · Solución 3 · Paso 5 de 8 · Teclado físico — El teclado del ordenador, otra vez

```
window.addEventListener('keydown', evento => {
  if (evento.ctrlKey || evento.altKey || evento.metaKey) return;
  const letra = evento.key.toUpperCase();
  if (ALFABETO.includes(letra)) this.jugar(letra);
});
```

S3-6 · Solución 3 · Paso 6 de 8 · Hitos 5 y 6 — Fin de partida con ventana y estadísticas

```
finPartida(res) {
  this.dom.teclado.querySelectorAll('.tecla').forEach(tecla => tecla.disabled = true);
  this.dom.btnPista.disabled = true;
  this.statsRepo.recordGame(res.victoria);

  if (res.victoria) {
    this.dom.modalIcono.textContent = '🏆';
    this.dom.modalTitulo.textContent = '¡Victoria!';
    this.dom.modalDesc.textContent = `Has adivinado "${res.palabraSecreta}".`;
  } else {
    this.dom.caraDerrota.classList.add('visible');
    const casillas = this.dom.palabra.querySelectorAll('.casilla-letra');
    casillas.forEach((casilla, i) => { casilla.textContent = res.palabraSecreta[i]; });
    this.dom.modalIcono.textContent = '💀';
    this.dom.modalTitulo.textContent = 'Derrota';
    this.dom.modalDesc.textContent = `La palabra era "${res.palabraSecreta}".`;
  }

  this.dom.modalRacha.textContent = this.statsRepo.stats.rachaActual;
  this.dom.modalVidas.textContent = res.maxFallos - res.fallos;
  setTimeout(() => this.dom.modalFin.classList.remove('oculta'), 400);
}
```

S3-7 · Solución 3 · Paso 7 de 8 · Los botones de la barra — Dificultad, sonido, pista y nueva partida

```
this.dom.selectDif.addEventListener('change', evento => {
  this.engine.dificultad = evento.target.value;
  this.iniciar();
});

this.dom.btnAudio.addEventListener('click', () => {
  const silenciado = this.audio.toggleMute();
  this.dom.iconoAudio.textContent = silenciado ? '🔇' : '🔊';
});

this.dom.btnPista.addEventListener('click', () => {
  this.dom.cajaPista.classList.remove('oculta');
  this.dom.btnPista.disabled = true;
});

this.dom.btnNueva.addEventListener('click', () => this.iniciar());
this.dom.modalBtn.addEventListener('click', () => this.iniciar());
```

S3-8 · Solución 3 · Paso 8 de 8 · Temporizador — 45 segundos de reloj en el modo Difícil

```
// ---- AhorcadoEngine ----
dificil: { maxFallos: 5, pistasGratis: false, tiempoMax: 45 }

tiempoAgotado() {
  this.juegoTerminado = true;
  const max = this.configDificultades[this.dificultad].maxFallos;
  return { letra: null, acierto: false, fallos: this.fallos, maxFallos: max,
           victoria: false, derrota: true, palabraSecreta: this.itemActual.palabra };
}

// ---- AhorcadoUI ----
// En el constructor:
this.temporizador = null;
this.inicioPartida = 0;
// En this.dom:
cajaTiempo: document.getElementById('caja-temporizador'),
tiempoTexto: document.getElementById('tiempo-texto'),
barraTiempo: document.getElementById('barra-tiempo-fill'),
modalTiempo: document.getElementById('modal-stat-tiempo'),

// Al final de iniciar():
this.inicioPartida = Date.now();
this.iniciarTemporizador();

iniciarTemporizador() {
  clearInterval(this.temporizador);
  const max = this.engine.configDificultades[this.engine.dificultad].tiempoMax;

  if (!max) {
    this.dom.cajaTiempo.classList.add('oculta');
    return;
  }

  let restantes = max;
  const pintar = () => {
    this.dom.tiempoTexto.textContent = `${restantes}s`;
    this.dom.barraTiempo.style.width = `${(restantes / max) * 100}%`;
  };

  this.dom.cajaTiempo.classList.remove('oculta');
  pintar();

  this.temporizador = setInterval(() => {
    restantes--;
    pintar();
    if (restantes <= 0) {
      clearInterval(this.temporizador);
      this.finPartida(this.engine.tiempoAgotado());
    }
  }, 1000);
}

// En finPartida(res): primera línea
clearInterval(this.temporizador);
// ... y en la rama de derrota:
this.dom.modalTitulo.textContent = res.letra === null ? '¡Tiempo agotado!' : 'Derrota';
// ... y junto a la racha y las vidas:
this.dom.modalTiempo.textContent = `${Math.round((Date.now() - this.inicioPartida) / 1000)}s`;
```



Fuentes y Documentación Oficial

1. MDN Web Docs — ¿Qué son las herramientas de desarrollo del navegador?
2. MDN Web Docs — Element.classList
3. MDN Web Docs — Set
4. MDN Web Docs — Clases
5. MDN Web Docs — Math.random()
6. MDN Web Docs — Math.floor()
7. MDN Web Docs — String.prototype.includes()
8. MDN Web Docs — Array.prototype.includes()
9. MDN Web Docs — Array.prototype.join()
10. MDN Web Docs — Node.textContent
11. MDN Web Docs — EventTarget.addEventListener()
12. MDN Web Docs — Document.getElementById()
13. MDN Web Docs — Element: keydown event
14. MDN Web Docs — KeyboardEvent: key property
15. MDN Web Docs — String.prototype.trim()
16. MDN Web Docs — String.prototype.toUpperCase()
17. MDN Web Docs — RegExp.prototype.test()
18. MDN Web Docs — Document.createElement()
19. MDN Web Docs — HTMLElement: dataset property
20. MDN Web Docs — Node.appendChild()
21. MDN Web Docs — Array.prototype.fill()
22. MDN Web Docs — Element.innerHTML
23. MDN Web Docs — Event.target
24. MDN Web Docs — Element.closest()
25. MDN Web Docs — Document.querySelector()
26. MDN Web Docs — Burbujeo de eventos
27. MDN Web Docs — Web Audio API
28. MDN Web Docs — Window.localStorage
29. MDN Web Docs — JSON
30. MDN Web Docs — this
31. MDN Web Docs — Funciones flecha
32. MDN Web Docs — HTMLElement: change event
33. MDN Web Docs — clearInterval()
34. MDN Web Docs — setInterval()
35. MDN Web Docs — Date.now()